

Methodological Protocol

This book is organized around a *claims-as-builds* protocol: each substantive claim is paired with an executable artifact, a test, or both. In practice, a theorem is not treated as complete until it is accompanied by a buildable representation of the statement, a reproducible derivation or proof sketch, and a checkable artifact that witnesses the intended behavior. This approach keeps the mathematical narrative and the implementation narrative aligned throughout the book.

The working protocol for each technical development is:

1. **Acquire** the relevant specifications, datasets, or formal statements.
2. **Calibrate** the error tolerance $\varepsilon(r)$ at the appropriate resolution or scale r .
3. **Log** the reproducibility metadata, including hashes, random seeds, toolchains, and any other build inputs needed to reconstruct the result.

The acquire step fixes the source material for a result: definitions, assumptions, input instances, and any external data or formal specifications. The calibrate step records the precision regime in which the claim is intended to hold, so that approximate statements, multi-resolution arguments, and robustness bounds can be compared on a common footing. The log step records enough provenance to make the result auditable and rerunnable, including versioned dependencies, compiler or interpreter settings, and the exact parameters used in experiments or checks.

Throughout the book, reproducibility is treated as part of the mathematical content rather than as an afterthought. When a chapter introduces a construction, the corresponding artifact should make clear what is being built, what is being tested, and what assumptions are required for the build to succeed. When a chapter proves an equivalence or closure property, the associated test should reflect the intended invariants at the relevant level of abstraction. In this sense, the protocol is not merely administrative: it is the mechanism by which claims are made inspectable.

The reading path is intentionally nonuniform. Readers with a stronger background in discrete mathematics or category theory may proceed directly into the foundational chapters, while readers who prefer a more concrete entry point may use the dependency structure as a guide and begin with the sections that emphasize examples, contracts, or case studies. The dependency graph across chapters is designed to support both styles: it identifies which results rely on earlier material and which chapters can be read with only the front matter and notation conventions in hand.

In particular, the book is arranged so that the early chapters establish the formal language, the middle chapters develop compositional constructions and proof-carrying artifacts, and the later chapters apply the same protocol to symmetry analysis, robustness, computation, and case studies. Readers are encouraged to treat the chapter dependencies as a map rather than a strict linear path: the map indicates where a result is used, what background it presupposes, and where a reader may safely branch depending on prior experience.

For practical use, the protocol can be summarized as follows. First, identify the claim and the artifact that should witness it. Second, determine the scale or tolerance at which the claim is meant to hold and record the corresponding $\varepsilon(r)$. Third, preserve the full build context so that the claim can be reconstructed later from the same inputs. This includes hashes for source files, seeds for randomized procedures, and the toolchain versions used to generate or verify the artifact. The result is a workflow in which proofs, tests, and build logs are all part of the same evidentiary chain.

The remainder of the book assumes this protocol implicitly. Claims should be read as buildable statements, constructions should be read as artifacts with interfaces and checks, and examples should be read as reproducible instances of the general theory. The chapter map should be used as

a dependency graph: follow it sequentially when you want the full development, or use it selectively when you want to enter through a concrete application and backfill the prerequisites as needed.

For orientation, the protocol also establishes a reading map across the book. The front matter supplies the notation and methodological commitments; the foundational chapters develop the abstract language of composition and closure; the middle chapters translate those ideas into modular constructions, feedback, and proof-carrying artifacts; and the later chapters apply the same discipline to symmetry, robustness, algorithms, and domain-specific case studies. Readers may therefore approach the book either top-down, by following the dependency graph chapter by chapter, or bottom-up, by starting from a concrete application and tracing the prerequisites backward until the relevant foundations are in place.

This reading map is meant to be practical rather than ceremonial. If a result depends on a prior definition, lemma, or convention, that dependency should be visible in the chapter structure and in the associated build artifacts. If a reader encounters a later chapter first, the dependency graph should indicate exactly which earlier material must be recovered before the claim can be checked or reused. In that sense, the book is not only a sequence of chapters but also a navigable system of dependencies, build steps, and reproducibility records.